

University Number:

--	--	--	--	--	--	--	--	--	--

THE UNIVERSITY OF HONG KONG

DEPARTMENT OF COMPUTER SCIENCE

Final Examination

COMP7404 Computational Intelligence and Machine Learning (Subclass B)

18 December 2023, 6:30 pm - 8:30 pm

Only approved calculators as announced by the Examinations Secretary can be used in this examination. It is the candidates' responsibility to ensure that their calculator operates satisfactorily, and candidates must record the name and type of the calculator used on the front page of the examination script.

Calculator Brand: _____

Calculator Model: _____

Answer all questions on this question paper in the space provided.

Write your University No. on every page and Seat No. below.

Seat No. _____

Question	Max Points	Your Points (examiner use only)
1	40	
2	10	
3	8	
4	8	
5	10	
6	12	
7	12	
Total	100	

Question 1 [40 pts total]

Indicate whether each of the following statements is true or false by writing T or F in the provided box. Each correct answer is worth 2 pts, while each incorrect answer will result in a deduction of 2 pts. No explanation is necessary.

1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20

1.1: A* Search is optimally efficient, meaning no other algorithm can find the optimal solution with fewer expansions.

1.2: The heuristic function used in A* Search must always be admissible and consistent.

1.3: The time complexity of A* Search is polynomial in the size of the graph.

1.4: Breadth-First Search (BFS) uses more memory compared to Depth-First Search (DFS) because it needs to store all the child nodes at the current level.

1.5: A* search can be considered as a special case of Greedy Best-First-Search.

1.6: In a graph without cycles, Depth-First Search (DFS) can get stuck in an infinite loop if it doesn't mark nodes as visited.

1.7: Depth-First Search (DFS) maintains a queue data structure for storing the nodes to be visited.

1.8: In CSPs, the degree heuristic selects the variable with the most constraints on remaining variables.

1.9: The minimum remaining values (MRV) heuristic in CSPs selects the variable with the most "legal" values left in its domain.

1.10: Alpha-Beta pruning can change the result obtained by the Minimax algorithm.

1.11: In adversarial search, the search space is typically represented as a game tree.

1.12: The Expectimax algorithm can be used in games that involve elements of chance or randomness.

1.13: In an MDP, the transition probabilities are dependent on all previous states.

1.14: The Bellman equation is used to compute the optimal value function in an MDP.

1.15: The value iteration algorithm for MDPs requires multiple passes over the state space until the value function converges.

1.16: The Q-value of a state-action pair in an MDP represents the expected return from starting in a given state, taking a specific action, and then following a particular policy.

University Number:

--	--	--	--	--	--	--	--	--	--

1.17: Reinforcement Learning is a type of supervised learning.

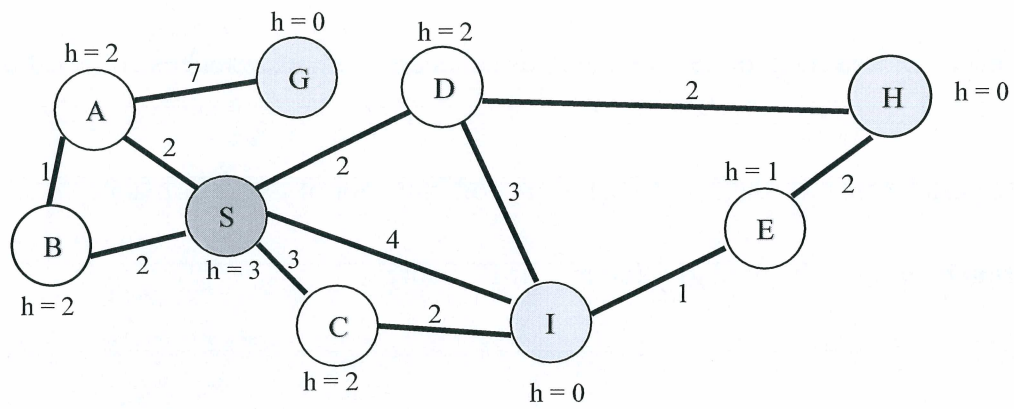
1.18: In Reinforcement Learning, off-policy methods can learn from experiences collected using a different policy.

1.19: A model's performance on the training set is a good indicator of its performance on unseen data.

1.20: A confusion matrix can be used to calculate the F1 Score.

Question 2 [10 pts total]

Consider the following search problem where S is the start state, and G, H and I are goal states.



2.1 (2 pts): Write down and explain if the provided heuristic is

- (a) admissible, and
- (b) consistent

University Number:

--	--	--	--	--	--	--	--	--	--

2.2 (8 pts): Assuming that children are added to the frontier in alphabetical order, write down

(a) the order of explored states, and

(b) the solution path

for each of the following search algorithms

(i) BFS-GSA, (ii) DFS-GSA, (iii) UCS-TSA, (iv) A*-GSA

--

Question 3 [8 pts total]

Consider the following Python code.

```
class Graph:
    def __init__(self):
        self.graph = {}

    def add_edge(self, u, v):
        if u not in self.graph: self.graph[u] = []
        self.graph[u].append(v)

    def r(self, start_node):
        v = set()
        self._r(start_node, v)

    def _r(self, node, v):
        if node not in v:
            print(node, end=' ')
            v.add(node)
        if node in self.graph:
            for n in self.graph[node]: self._r(n, v)
```

3.1 (4 pts): Name the algorithm that this code implements.

3.2 (4 pts): Consider the following Python code.

```
g = Graph()
g.add_edge(0, 1)
g.add_edge(0, 2)
g.add_edge(1, 2)
g.add_edge(2, 0)
g.add_edge(2, 4)
g.add_edge(2, 3)
g.add_edge(2, 1)
g.add_edge(3, 3)
g.add_edge(4, 3)
start_node = 2
g.r(start_node)
```

Draw the graph and write down the output of the program. What does the output represent?

--	--	--	--	--	--	--	--	--	--

Question 4 [8 pts total]

Consider a Markov Decision Process (MDP) with three states $\{S_0, S_1, S_2\}$ and two possible actions $\{A_0, A_1\}$ in each state. The agent starts in state S_0 and the episode ends when it reaches state S_2 . Here are the transition probabilities and rewards:

- In state S_0 , taking action A_0 will always lead to state S_1 with a reward of +5. Taking action A_1 will also always lead to state S_1 , but with a reward of +3.
- In state S_1 , taking action A_0 will lead back to state S_0 with a probability of 0.8 (reward +2) and to state S_2 with a probability of 0.2 (reward +10). Taking action A_1 will lead to state S_2 with a probability of 1.0 (reward +1).
- State S_2 is a terminal state.
- The agent uses discount factor $\gamma = 0.9$. Assume the agent has initial Q-values all equal to zero.

4.1 (2 pts): If the agent uses Q-learning what will be the updated Q-values after the first episode, assuming the agent starts with action A_0 in state S_0 , then chooses action A_0 in state S_1 , and ends up in state S_2 ?

--

4.2 (3 pts): Continuing from question 4.1, what will be the updated Q-values after the second episode, where the agent starts with action A_1 in state S_0 , then chooses action A_1 in state S_1 , and ends up in state S_2 ?

--

4.3 (3 pts): Continuing from question 4.2, based on the updated Q-values, what will be the agent's next action for each of the states? Assume the agent is employing the Q-learning algorithm with an ϵ -greedy exploration strategy with $\epsilon=0$.

--

Question 5 [10 pts total]

5.1 (5 pts): The Perceptron algorithm makes a key assumption about the data before training. What is this assumption and what happens when this assumption is violated?

5.2 (5 pts): Your Adaline model seems to be overfitting when allowed to train until completion. Can you elaborate on whether early stopping could be a potential solution to this issue?

--	--	--	--	--	--	--	--	--	--

Question 6 [12 pts total]

Consider the following Python code snippet.

```
from sklearn import datasets
from sklearn.model_selection import train_test_split
from sklearn import svm
from sklearn.metrics import accuracy_score
from sklearn.preprocessing import StandardScaler

iris = datasets.load_iris()

X_train, X_test, y_train, y_test = train_test_split(iris.data, iris.target, test_size=0.2,
random_state=42)

sc = StandardScaler()
X_train = sc.fit_transform(X_train)
X_test = sc.transform(X_test)

clf = svm.SVC(kernel='linear', C=1.0, random_state=42)

clf.fit(X_train, y_train)

y_pred = clf.predict(X_test)

print("Accuracy:", accuracy_score(y_test, y_pred))
```

6.1 (2 pts): What model does this code implement?

--

6.2 (2 pts): Explain the purpose of the StandardScaler in this code and why is it important in this context?

--

6.2 (4 pts): Modify the code to perform hyperparameter tuning using sklearn's GridSearchCV function on the parameter C. Explain the purpose of this parameter and why it is necessary to hyperparameter tune it. When grading, we will overlook minor syntax mistakes in your code.

6.3 (4 pts): The provided model is sensitive to outliers. Explain why, suggest and implement in the code a robust method for handling outliers in the dataset before training the model.

--	--	--	--	--	--	--	--	--	--

Question 7 [12 pts total]

7.1 (4 pts): Provide an example of four hyperparameters in a Random Forest that you would consider tuning.

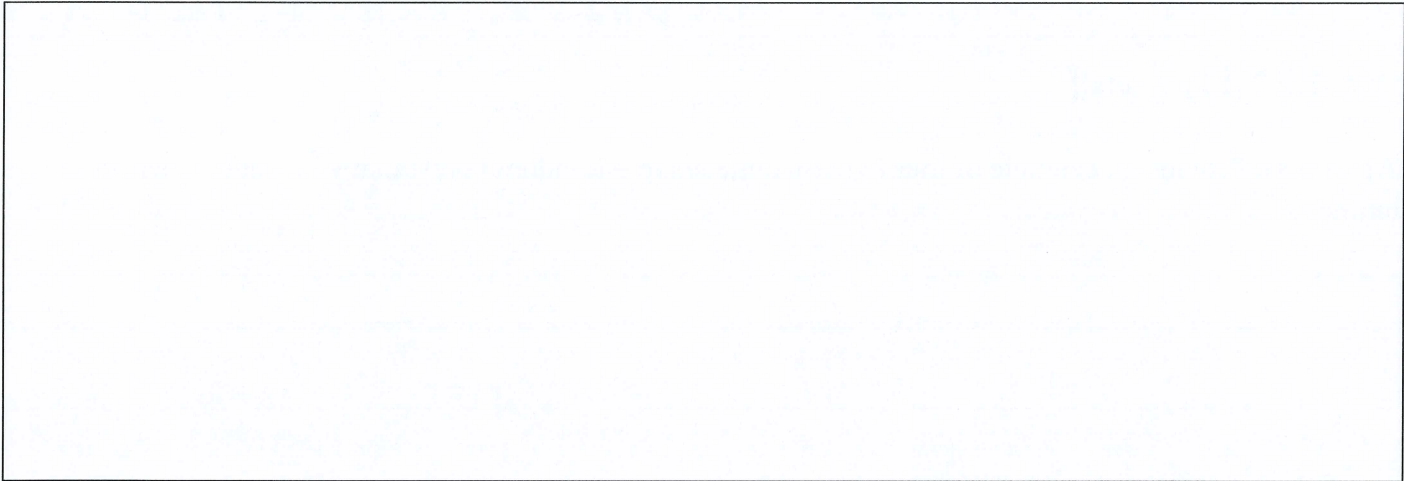
--

7.2 (4 pts): Random search for hyperparameter tuning is a method where random combinations of the hyperparameters are used to find the best solution for the built model. Discuss the use of grid search and random search for hyperparameter tuning. What are the advantages and disadvantages of each method?

--

7.3 (4 pts): Consider that you are working with a highly imbalanced binary classification task. You decide to use an ensemble of different classifiers including Logistic Regression, Support Vector Machines (SVM), and a Random Forest Classifier to improve prediction performance. Explain how cross-validation and hyperparameter tuning can be used in conjunction with each other to achieve a better bias-variance trade-off with your ensemble model.

--



END OF PAPER